

AMR Final Project

Julia Töws, Rene Hadder, Tim Dörpinghaus

Abstract—In this project, we brought three navigation functionalities from the course Autonomous Mobile Robots to the Robile platform: path and motion planning with A* and a potential field path planner, Monte Carlo localisation with our own particle filter, as well as fringe-based exploration together with SLAM-gmapping. All functionalities work reliably in the simulation: the robot reaches its goals, localises itself globally in a known map and explores an unknown environment completely. On the real robot, we tested the exploration and had to solve several additional problems, such as different clock times of the laptop and the robot, invalid laser measurements, collisions with the corners of the robot and a narrow passage in our test arena. To handle these problems without many special cases, we replaced our navigator by a potential field in the configuration space of the robot, with which the robot also passed the narrow passage and mapped the whole arena.

Index Terms—Mobile robot navigation, Monte Carlo localisation, frontier-based exploration

I. PROJECT DESCRIPTION

A. Motivation

During the course Autonomous Mobile Robots, we implemented several basic navigation methods, but mostly in simulation. The motivation of this project is to show what we have learned in the course and to bring these methods to a real robot. This gives us experience with a realistic project, where the robot, its sensors and the environment do not behave as ideally as in the simulation.

B. Problem Statement

The task is to deploy three functionalities from the course on the Robile platform, a four-wheeled omnidirectional robot with a laser scanner at its front. The three parts build on each other:

- 1) **Path and motion planning:** The potential field planner from the assignments has to be ported to the real robot. It is combined with a global path planner (e.g. A*), which finds a rough path of waypoints on a map. The potential field planner then moves the robot from waypoint to waypoint and avoids obstacles.
- 2) **Localisation:** We have to implement our own particle filter (Monte Carlo localisation) that estimates the pose of the robot in an existing map.
- 3) **Environment exploration:** The robot has to explore an unknown environment on its own. An exploration component selects the poses to explore, ideally at the map fringe (the boundary between the explored and the

unexplored region), and a SLAM component creates the map.

All functionalities first had to work in the simulation and then on the real robot.

C. Proposed Approach

We implemented all parts as nodes of one ROS 2 (Humble) package in Python:

- **A* path planner:** A* searches the shortest path on the occupancy grid map. The obstacles are grown by the robot radius (configuration space), so that the path keeps a distance to walls. The path is sampled into waypoints, which are sent to the navigator one after the other.
- **Potential field navigator:** The waypoint attracts the robot, and the obstacles measured by the laser scanner repel it. The potential field is defined over the position and the orientation of the robot, and its negative gradient gives the velocity of the omnidirectional robot.
- **Particle filter:** Many particles represent possible poses of the robot. They are moved with the odometry (motion model), weighted by comparing the laser scan with the map (measurement model) and resampled according to their weights.
- **Fringe-based explorer:** The explorer selects the closest reachable cell at the map fringe as the next goal. The map itself is created by *slam_gmapping*, an existing SLAM implementation based on a particle filter.

II. DESCRIBING THE CODE

This section describes the implemented nodes and the functionalities we added to the basic algorithms. The source code and a more detailed description of all topics and parameters are available in the README of our repository¹.

A. Task 1: Path and Motion Planning

Path planner. The node *path_planner* receives a goal and the occupancy grid map. A* searches the shortest path from the robot to the goal, using the direct and diagonal neighbours of a cell and the Euclidean distance as heuristic. Occupied cells are grown by 0.35 m, so that the path keeps a distance to obstacles. If no path is found, e.g. in a narrow passage, this distance is reduced step by step, so that the path still stays in the middle of the passage. The path is sampled into waypoints, and only one waypoint at a time is sent to the navigator. Intermediate waypoints only have to be passed within 0.3 m, while the goal has to be reached within 0.1 m.

Potential field navigator. The node *potential_field_navigator* moves the robot to the current waypoint.

*Submitted to the Department of Computer Science at Hochschule Bonn-Rhein-Sieg in partial fulfilment of the requirements for the degree of Master of Science in Autonomous Systems

[‡]Submitted in September 2026

¹<https://github.com/t1md0e/amr-team-jrt>

The potential field is defined in the configuration space $\mathbf{q} = (x, y, \theta)$ of the robot, i.e. over its position and orientation:

$$U(\mathbf{q}) = U_{att}(\mathbf{q}) + U_{rep}(\mathbf{q}), \quad \mathbf{F}(\mathbf{q}) = -\nabla U(\mathbf{q}). \quad (1)$$

Every obstacle point i of the laser scan has a repulsive potential, which depends on the clearance $d_i(\mathbf{q})$ between the point and the rectangular footprint of the robot ($0.76 \text{ m} \times 0.47 \text{ m}$) at the pose $\mathbf{q}$:

$$U_{rep}(\mathbf{q}) = \sum_i \frac{1}{2} \eta \left(\frac{1}{d_i(\mathbf{q})} - \frac{1}{\rho_0} \right)^2 \quad \text{for } d_i < \rho_0, \quad (2)$$

with $\eta = 0.0005$ and $\rho_0 = 0.4 \text{ m}$. The laser points are thinned out to one point per 5 cm cell, so that a wall counts by its length and not by the number of laser points. The attractive potential U_{att} pulls the robot to the waypoint (quadratic within 0.3 m , so that the robot slows down there, and conic beyond) and turns it towards the waypoint, so that the laser scanner looks in the driving direction. At the waypoint, it turns the robot to the orientation of the waypoint. The gradient is calculated numerically. As the robot is omnidirectional, $\mathbf{F}$ is directly used as velocity in x , y and θ , limited to the maximum speeds.

Compared to the basic potential field planner from the assignments, we added:

- **Rectangular footprint and orientation:** As the clearance is measured from the footprint at the pose $\mathbf{q}$, a rotation that brings a corner closer to a wall increases U and is avoided. In a narrow passage, the repulsions of both sides cancel each other in the middle.
- **Line search:** A step along $\mathbf{F}$ is only taken if U decreases, otherwise a shorter step is tried. This prevents the robot from oscillating between the walls of a narrow passage.
- **Local minima:** If the robot gets no closer to the waypoint for 5 s , it escapes with a random walk, as proposed in the lecture.
- **Obstacle memory:** The laser scanner only looks to the front. Therefore, laser points are remembered for some time, so that obstacles next to and behind the robot are still known. This is also known as spatial memory.

B. Task 2: Localization

The node *particle_filter* implements Monte Carlo localisation with 2000 particles. When the map is received, the particles are spread uniformly over all free cells (global localisation). Alternatively, a start pose can be set in RViz, and the particles are sampled around it. The filter is only updated after the robot has moved or rotated a little (0.05 m or 0.05 rad). Every update has three steps:

- 1) **Motion update:** Every particle is moved with the odometry motion model (rotation, translation, rotation), with Gaussian noise added on every part.
- 2) **Measurement update:** For 30 laser beams, the expected range is calculated for every particle by casting a ray through the map. The weight of a particle is the likelihood of the measured ranges (Gaussian with $\sigma = 0.4 \text{ m}$

around the expected range, plus a small probability for noise).

- 3) **Resampling:** Particles are drawn proportionally to their weights.

The estimated pose is the weighted mean of the particles. The node publishes it and the transform from the map to the odometry frame, which the path planner and the navigator uses.

We added the following extensions:

- **Recovery (augmented MCL):** If the short-term average of the particle weights drops below the long-term average, random particles are added. This helps if the robot gets lost or was kidnapped.
- **Symmetric environments:** The particles are not resampled before the robot has moved, as a single scan often fits several places of a symmetric map.
- **Partial maps:** Beams that end in an unknown cell of the map will get a uniform likelihood, so that the filter also works with maps that are not completely explored.

C. Task 3: Environment Exploration

The map is created by *slam_gmapping*, a ROS 2 port of the gmapping SLAM package, which also publishes the pose of the robot in the map. Our node *explorer* selects the goals:

- 1) It finds the fringe cells, i.e. free cells next to unknown cells, and groups them into connected regions. Regions with less than 4 cells are ignored.
- 2) Occupied cells are grown by 0.5 m , so that the robot fits inside the goal and can rotate there. If a fringe cell is closer to obstacles, e.g. next to a wall, the goal is a *view pose*: the closest free pose within 1 m , from which the robot looks at the fringe cell.
- 3) A wavefront search (breadth-first search over the free cells) finds the closest reachable goal that is at least 1 m away. This goal is sent to the path planner.
- 4) The goal orientation points towards the unknown cells around the goal, so that the laser scanner scans the unexplored region once the robot arrives.

A new goal is selected when the goal is reached, when the region around it has already been explored on the way, or when the robot makes no progress. Goals without progress and fringe cells that are still unknown after the robot has reached their view pose are put on a blacklist.

We added the following functionalities:

- **Initial step:** At the start, the cell of the robot itself is still unknown, because the laser scanner is at the front. Therefore, the robot first moves a small step forward. A small enclosed unknown region at the start position, i.e. the floor below the robot that the laser scanner never sees, is ignored.
- **Driving direction:** If the goal was already explored on the way, the next goal is searched close to the old goal. Otherwise, the robot would turn around, e.g. inside a narrow passage, as soon as it sees into the room behind it.
- **Stopping and saving:** The exploration may be stopped after a maximum time or if no new area is explored for

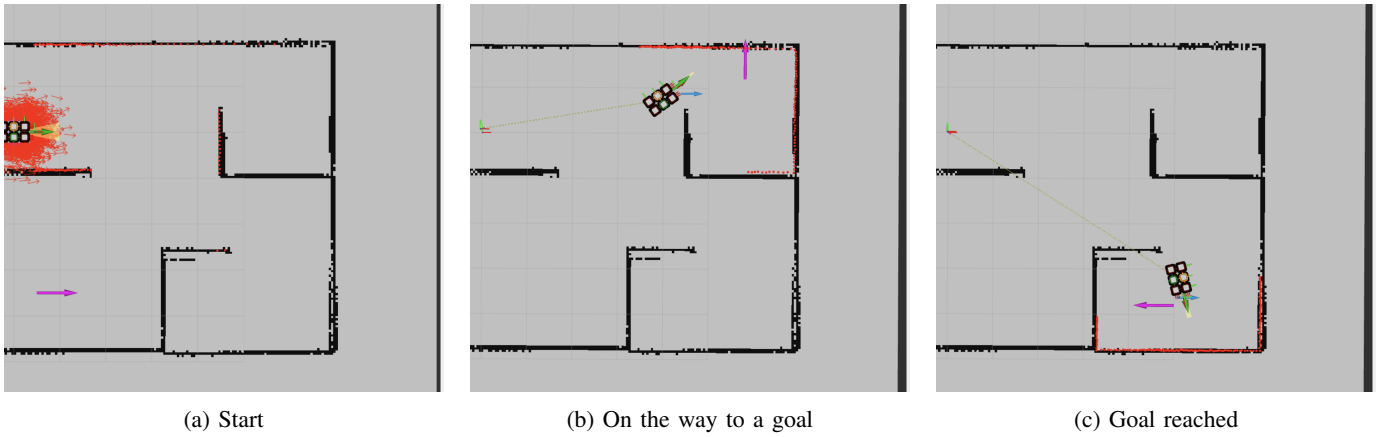


Fig. 1: Task 1 in the simulation (RViz): The robot follows the waypoints of the A* path to the goal (purple arrow).

some time. The map gets saved regularly, so that it can be used for the localisation algorithm in task 2.

III. EXECUTING THE CODE

We first tested all tasks in the simulation. On the real robot, we tested the exploration (task 3). This section describes both environments and shows the results.

A. The Simulation

Environment. We used the Robile simulation in Gazebo Classic with ROS 2 Humble. It ran in Docker containers: partly on an Ubuntu computer in a container with Ubuntu, and mostly on a Mac, where the container emulates the amd64 architecture, because Gazebo Classic is not available for ARM processors. As the Gazebo interface did not open in the container, we only ran the simulation server and used RViz to see the robot, the laser scan, the map and the particles. Most tests were done in the world *closed_walls* (about $7\text{ m} \times 7\text{ m}$) and some in a house world as a harder test with open doors.

Task 1: Path and motion planning. For this task, the robot was localised with our particle filter in a map created before. We sent three goals one after the other (Fig. 1). The robot reached all goals without collisions in about 30 s to 50 s each, with a final position error of about 0.02 m to 0.16 m. With the new potential field in the configuration space (Sec. IV-C), the robot also reached all three goals, with an error of about 0.03 m to 0.14 m. Video: *aufgabe1 navigation*

Task 2: Localisation. We started the particle filter without a start pose, so the particles were spread over the whole map (Fig. 2). While the robot was moving, the particles converged to the correct pose after about 15 s to 90 s, depending on the start position. After that, the error of the estimated position was about 0.01 m to 0.15 m. In some runs, the filter first converged to a wrong but similar-looking place and corrected itself after 30 s to 80 s thanks to the random particles. Video: *aufgabe2 lokalisierung*

Task 3: Environment exploration. The robot started in an unknown environment and explored it on its own (Fig. 3). In the final runs in *closed_walls*, the complete map (about 52 m^2) was created in about 90 s to 120 s, and the exploration stopped

by itself when no reachable fringe was left. Video: *aufgabe3 exploration*

B. The Real Robot

Environment. In the lab, we used a Robile with four wheels and a laser scanner at its front. The ROS 2 nodes ran in a Docker container with Ubuntu on a laptop with Ubuntu, which was connected to the robot via WLAN. We built an arena in our lab room (Fig. 4): an area of about $4\text{ m} \times 6\text{ m}$, bounded by boxes, cupboards, shelves and a sofa, with a narrow passage between two parts of the area. Compared to the simulation, we reduced the speed of the robot to at most 0.3 m/s and 0.8 rad/s.

Real world explorer. The robot started in the arena without a map and explored it with the explorer and *slam_gmapping* (Fig. 5). In the first tests, the robot did not pass the narrow passage to the second part of the arena (see Sec. IV-C). With the new navigator and the improved explorer, the robot drove through the narrow passage and mapped the whole arena. Videos: *Mapping in Lab; Real Robot, Mapping in Lab; Screenrecording*

IV. CHALLENGES

This section describes the biggest problems we faced during the project and how we solved them.

A. Simulation

Setup. We ran the simulation in Docker containers: some simulations on an Ubuntu computer in a container with Ubuntu and ROS 2, and most of them on a Mac. On the Mac, the container had to emulate the amd64 architecture, which caused several problems: the Gazebo interface did not open, RViz sometimes froze, and older processes were still running, which interfered with new runs. We therefore only used the Gazebo server with RViz and restarted the container before every test. In addition, the ROS 2 port of gmapping sometimes stopped after the first scan, so we restarted it automatically in this case.

Navigation. At first, the robot drove in circles around the waypoints or got stuck in front of walls. The reason was that the repulsive forces of all laser points were summed up, which

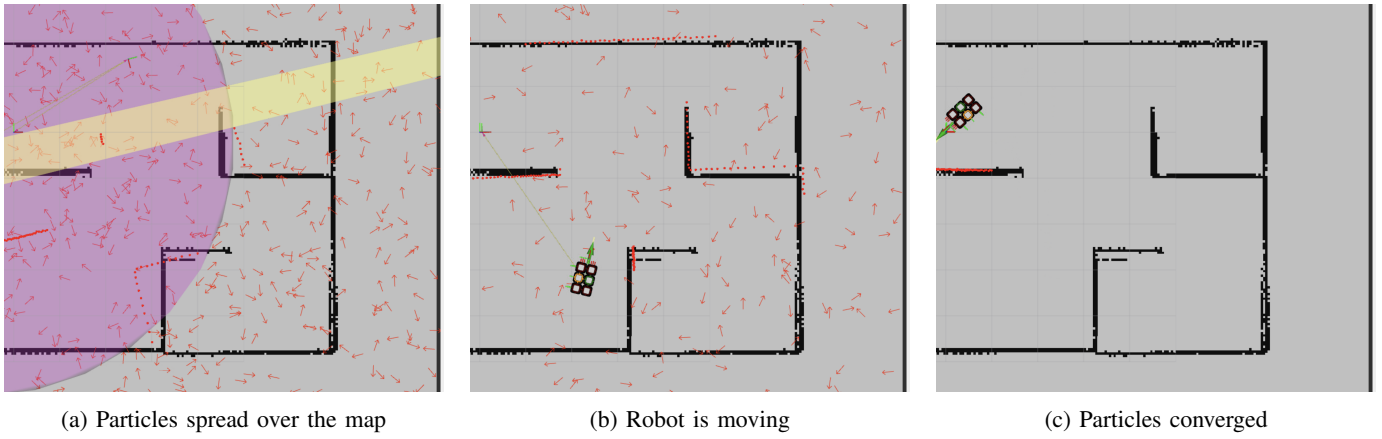


Fig. 2: Task 2 in the simulation: global localisation with the particle filter (red arrows are particles).

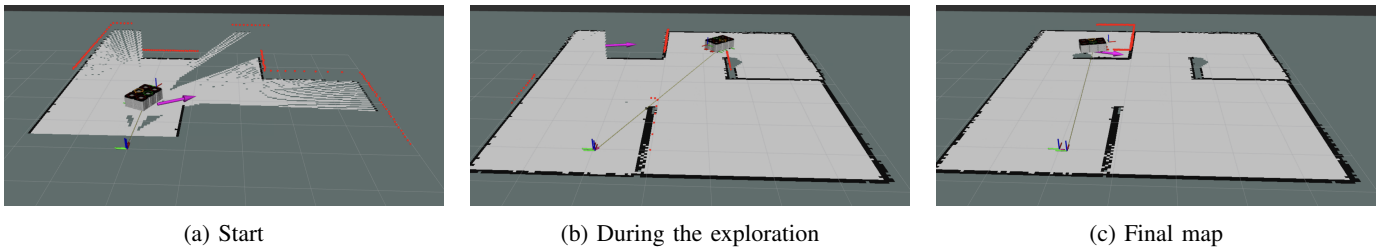


Fig. 3: Task 3 in the simulation: The map created by gmapping grows while the explorer selects goals at the map fringe.



Fig. 4: The Robile in our arena in the lab room, bounded by boxes and furniture.

made the repulsion of a wall 10 to 20 times stronger than the attraction. We solved this by using only the closest obstacle, by slowing down close to the waypoint and by rotating in place if the waypoint is to the side or behind the robot. Planning the path in the configuration space kept the path away from walls. Later, we replaced this navigator by a potential field in the configuration space (Sec. IV-C). We also found that the simulated robot hardly moves at commands below 0.25 m/s and 0.5 rad/s, so that small corrections of the navigator had no effect. This speed is now added to every command in the simulation.

Localisation. In symmetric environments, the particle filter

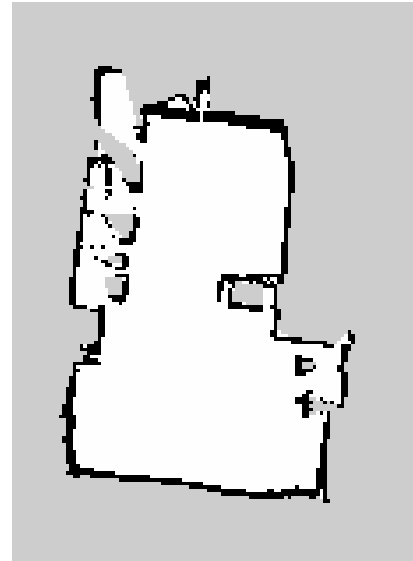


Fig. 5: Map of the arena created by the real world explorer (white: free, black: occupied, grey: unknown).

sometimes converged to a wrong place, because a single scan fits several places of the map. It helped to resample only after the robot had moved, to use more particles (2000 instead of 500) and a larger measurement noise ($\sigma = 0.4$ m instead of 0.2 m). Look-alike rooms are still a weak point, but the random particles usually correct a wrong estimate after some time.

Exploration. The explorer first chooses goals very close to the robot, so the robot moved slowly along the fringe, and in the house world it left the building through an open

door. We added a minimal goal distance of 1 m, an optional exploration boundary and a stopping criteria. We also found, that the explorer and the path planner converted coordinates to map cells differently, which resulted in failed path searches.

B. Connecting to the Robot

Ubuntu laptop. With the Ubuntu laptop, the robot did not move at first. One reason was that our navigator expected the name of the laser frame from the simulation (*base_laser_front_link*), while it is called *base_laser* on the real robot. Now, the frame is taken from the scan messages. The second reason was that the clock of the laptop was about 59 s ahead of the clock of the robot. Because of the different timestamps, the transforms between the frames could never be found, and the explorer did not send any goals. After we set the clock of the laptop back by this difference, the robot started moving.

macOS and Docker. We also tried to control the robot via a Mac-book, where ROS 2 runs in a Docker container. Docker on macOS runs the containers in a virtual machine, which blocks the automatic discovery of ROS 2 (multicast). We configured the communication to send messages directly to the robot (unicast). The robot and the container found each other, but no topics have arrived. The reason is that Docker on macOS drops network packets larger than about 1400 bytes, which is the case for most ROS 2 messages. A relay program that forwarded the packets did not solve the problem either, so we went back to the Ubuntu laptop.

C. From the Simulation to the Real Robot

Sensor data. The real laser scanner reports invalid measurements as 0, which created huge repulsive forces. These measurements are now ignored, by removing those outliers. The range of the real scanner (up to 60 m) is also much larger than in the simulation, so we limited the ray casting of the particle filter to 10 m.

Collisions with the corners. During a rotation close to a wall, the robot sometimes collided with the wall with one of its corners, because the navigator treated the robot as a one dimensional point. We added a rectangular footprint of the robot: the clearance is measured from the footprint, and the robot now only rotates with enough clearance.

Narrow passage. In our arena, the robot did not pass the narrow passage (about 0.8 m wide, estimated from the laser data). It turned left and right in front of it, moved back and tried again, until the explorer gave up the goal. The reason was the repulsion of both sides of the passage, which was much stronger than the attraction of the waypoint and turned the robot away. We rebuilt the arena in the simulation and added a special mode, in which the robot moves straight through the passage without rotating, and the path now stays in the middle of the passage. We also found that the obstacle memory of the navigator almost never worked, because the laser scan was a few milliseconds newer than the odometry.

Simplified navigator. For every problem, we had added a new special case to the navigator: a check of the rotation with the footprint, keeping the direction of a rotation, a detection of

blocked rotations, moving back along the driven path, moving away from obstacles and the mode for narrow passages. The code became complex, and the special cases interfered with each other: e.g. the passage mode was active almost all the time and slowed the exploration down so much that the map was not complete after 300 s. We therefore replaced all special cases by one potential field in the configuration space (x, y, θ) (Sec. II-A), which considers the shape and the orientation of the robot. The navigator became about 180 lines shorter. In the simulation, the exploration was complete after about 90 s again, and the robot passed the narrow passage of the rebuilt arena.

Exploration in the lab. With the new navigator, the robot still stayed in front of the passage, but this time because of the explorer. The only goal it selected was an unknown spot at the start position: the floor below the robot, which the laser scanner at the front never sees. The robot drove around it again and again. The fringe in and behind the passage was never selected, because it was closer than 0.5 m to obstacles or too small. We changed the explorer: such fringe cells are now explored from a view pose nearby, small regions from 4 cells on are used, and the unknown floor at the start position is ignored. In the simulation, the robot then drove through the passage and explored the whole arena in about 60 s, and on the real robot, it passed the narrow passage and mapped the whole arena as well.

D. Time Management

Our work depended on parts of the project that were planned to be done by one team member. As these parts were not done, the whole project was delayed, and most of the work had to be done shortly before the deadline. This was one reason why we relied more heavily on an AI assistant (see Appendix) than we initially intended to.

V. CONCLUSIONS

We implemented path and motion planning, localisation and exploration for the Robile and combined them in one ROS 2 package. In the simulation, all three tasks worked well: the robot reached its goals with an error of about 0.1 m, the particle filter found the pose of the robot without a starting pose, and the explorer created a complete map of the environment in about two minutes.

The step from the simulation to the real robot was harder than expected. Most problems were not in the algorithms themselves, but in the details: frame names, clock times, invalid sensor data, the real size of the robot and the network setup. The main lessons we learned are:

- The real size and shape of the robot matters. A robot that is modelled as a point collides with its corners.
- A simulation environment should be as similar as possible to the real test environment. Rebuilding our arena in the simulation helped us to find the causes of the problems with the narrow passage.
- Logs and visualisation (RViz) are essential to find the culprits of problems, especially on the real robot.

- Adding a special case for every problem makes the code complex, and the special cases interfere with each other. A potential field in the configuration space, which considers the shape and orientation of the robot, solved most of these problems with less code.

STATEMENT OF ORIGINALITY

I, the undersigned below, declare that this work has not previously been submitted to this or any other university and that it is, unless otherwise stated, entirely my own work. The report was, in part, written with the help of the AI assistants Claude Code and ChatGPT as described in the appendix. I am aware that content generated by AI systems is no substitute for careful scientific work, which is why all AI-generated content has been critically reviewed by me, and I take full responsibility for it.

_____	_____
Date	Signature
_____	_____
Signature	Signature

APPENDIX

Use of AI Assistants

We used AI assistants during the whole project: mainly Claude Code (Anthropic), and in addition ChatGPT (OpenAI) for help with coding and debugging in task 1. Claude Code was used for the following parts:

- **Code:** Claude Code helped writing the particle filter (task 2), the explorer (task 3) and the launch files, based on the methods of the course. It also made parts of the later fixes to the path planner and the potential field navigator (task 1), which were written by our team, and finally rewrote the navigator as a potential field in the configuration space.
- **Debugging:** We gave Claude Code the logs of the simulation and of the real robot, and it analysed them and suggested fixes, e.g. for the clock difference, the corner collisions and the narrow passage.
- **Simulation and tests:** Claude Code set up the Docker simulation, wrote test scripts that measured e.g. the explored area and the localisation error, and ran the tests in the simulation.
- **Git:** It created branches, commits and pull request descriptions. All commits made with its help are marked with “Co-Authored-By: Claude”.
- **Documentation:** It wrote parts of the README and a first version of this report, based on bulletpoints, pre-drafted passages, the README, the code and our previous chats. We reviewed and adapted the text.

Typical prompts were descriptions of the task or the problem, architectural requirements, an outline of the desired solution together with the logs of the robot. All results were checked by us in the simulation or on the real robot.